

Life Satisfaction Prediction using GDP per Capita

Assignment note

This assignment is not intended to be a full project. It serves as a follow up to reinforce the concepts we have covered in lecture so far. More comprehensive analyses and detailed projects will be introduced as we progress further into the course.

Learning objectives

- Load and visualize real-world data.
- Train and compare regression models using Scikit-Learn.
- Interpret model outputs and discuss generalization.

Imports

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
```

Part 1: Data Exploration and Visualization (20 marks)

Q1. Load the dataset `lifesat.csv` and display the first 5 rows (5 marks)

```
In [2]: df = pd.read_csv("./lifesat.csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

	Country	GDP per capita (USD)	Life satisfaction
0	Russia	26456.387938	5.8
1	Greece	27287.083401	5.4
2	Turkey	28384.987785	5.5
3	Latvia	29932.493910	5.9
4	Hungary	31007.768407	5.6

```
In [4]: df = df.rename(columns={
    "Country": "country",
    "GDP per capita (USD)": "gdp_per_capita",
    "Life satisfaction": "life_satisfaction"
})
```

Q2. Print basic info and summary statistics (5 marks)

```
In [5]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 27 entries, 0 to 26
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   country                27 non-null    str
1   gdp_per_capita         27 non-null    float64
2   life_satisfaction      27 non-null    float64
dtypes: float64(2), str(1)
memory usage: 780.0 bytes
```

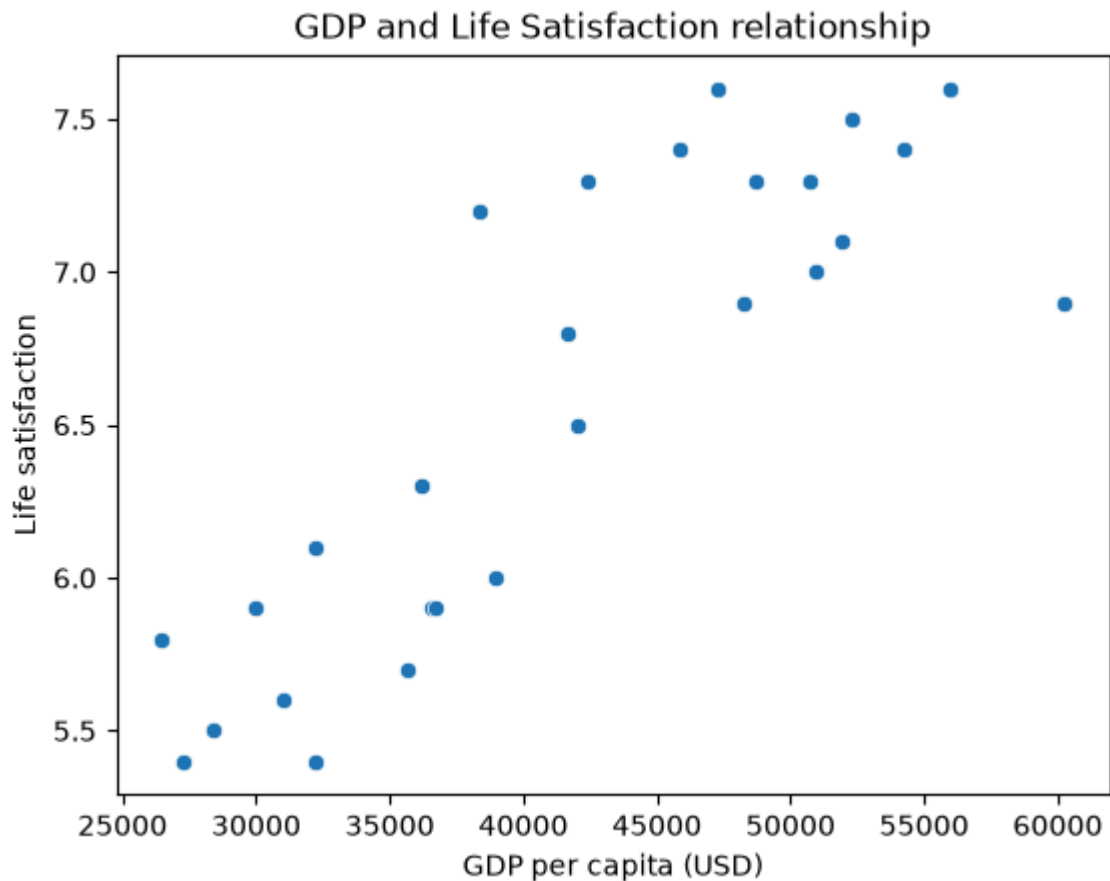
```
In [6]: df.describe()
```

```
Out[6]:
```

	gdp_per_capita	life_satisfaction
count	27.000000	27.000000
mean	41564.521771	6.566667
std	9631.452319	0.765607
min	26456.387938	5.400000
25%	33938.289305	5.900000
50%	41627.129269	6.800000
75%	49690.580269	7.300000
max	60235.728492	7.600000

Q3. Display a scatter plot for GDP per capita vs Life Satisfaction. Add labels, title, and discuss the observed relationship (10 marks)

```
In [7]: plt.title("GDP and Life Satisfaction relationship")
plt.xlabel("GDP per capita (USD)")
plt.ylabel("Life satisfaction")
sns.scatterplot(
    data=df,
    x=df["gdp_per_capita"],
    y=df["life_satisfaction"]
);
```



Observed relationship

The **higher GDP per capita**, the **higher Life satisfaction**. It represents positive correlation relationship between those 2 variables (features).

Part 2: Linear Regression Model (30 marks)

Q4. Extract input (X) and target (y). Print their shapes (5 marks)

```
In [8]: X = df[["gdp_per_capita"]]
        y = df["life_satisfaction"]
        print(X.shape, y.shape)
```

(27, 1) (27,)

```
In [9]: X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

```
In [10]: print(X_train.shape, y_train.shape)
```

(20, 1) (20,)

```
In [11]: print(X_test.shape, y_test.shape)
```

(7, 1) (7,)

Q5. Train a Linear Regression model & Display coefficient and intercept (10 marks)

```
In [12]: model = LinearRegression()
```

```
In [13]: model.fit(X_train, y_train)
```

```
Out[13]: ▼ LinearRegression ⓘ ?
```

► Parameters

► Fitted attributes

```
In [14]: print(model.coef_[0], model.intercept_)
```

6.219973474996385e-05 3.936302711267398

Q6. Plot the predicted regression line from the model along with a scatter plot of the data (10 marks)

```
In [15]: y_pred = model.predict(X_test)
```

```
In [16]: plt.title("GDP and Life Satisfaction Linear regression model")
plt.xlabel("GDP per capita (USD) - X")
plt.ylabel("Life satisfaction - y")
sns.lineplot(
    x = X_test.squeeze(axis = 1),
    y = y_pred
);
sns.scatterplot(
    data=df,
    x=df["gdp_per_capita"],
    y=df["life_satisfaction"]
);
```



Q7. Predict Life Satisfaction for GDP = 37,655.2 USD. Comment on result (5 marks)

```
In [17]: y_pred_linear = model.predict(pd.DataFrame(
        {
            "gdp_per_capita": [37_655.2]
        })
    )
    y_pred_linear[0]
```

```
Out[17]: np.float64(6.2784461632242365)
```

The expected **life satisfaction** in scale from 0 to 10 for country with **GDP of 37_655.2** is **6.28**

Part 3: K-Nearest Neighbors Regression (25 marks)

Q8. Train a KNeighborsRegressor (n_neighbors=3) (5 marks)

```
In [18]: knn_model = KNeighborsRegressor(n_neighbors=3)
```

```
In [19]: knn_model.fit(X_train, y_train)
```

```
Out[19]: ▼ KNeighborsRegressor ⓘ ?
```

▶ Parameters
 ▶ Fitted attributes

Q9. (10 marks)

- Predict Life Satisfaction for GDP = 37,655.2 USD

```
In [20]: y_pred_knn = knn_model.predict(pd.DataFrame(
        {
            "gdp_per_capita": [37_655.2]
        })
        y_pred_knn[0]
```

```
Out[20]: np.float64(5.933333333333334)
```

- Compare with Linear Regression

Compared to the **Linear regression** model, **KNN** predicted a lower value for life satisfaction: **5.93**

Q10. (10 marks)

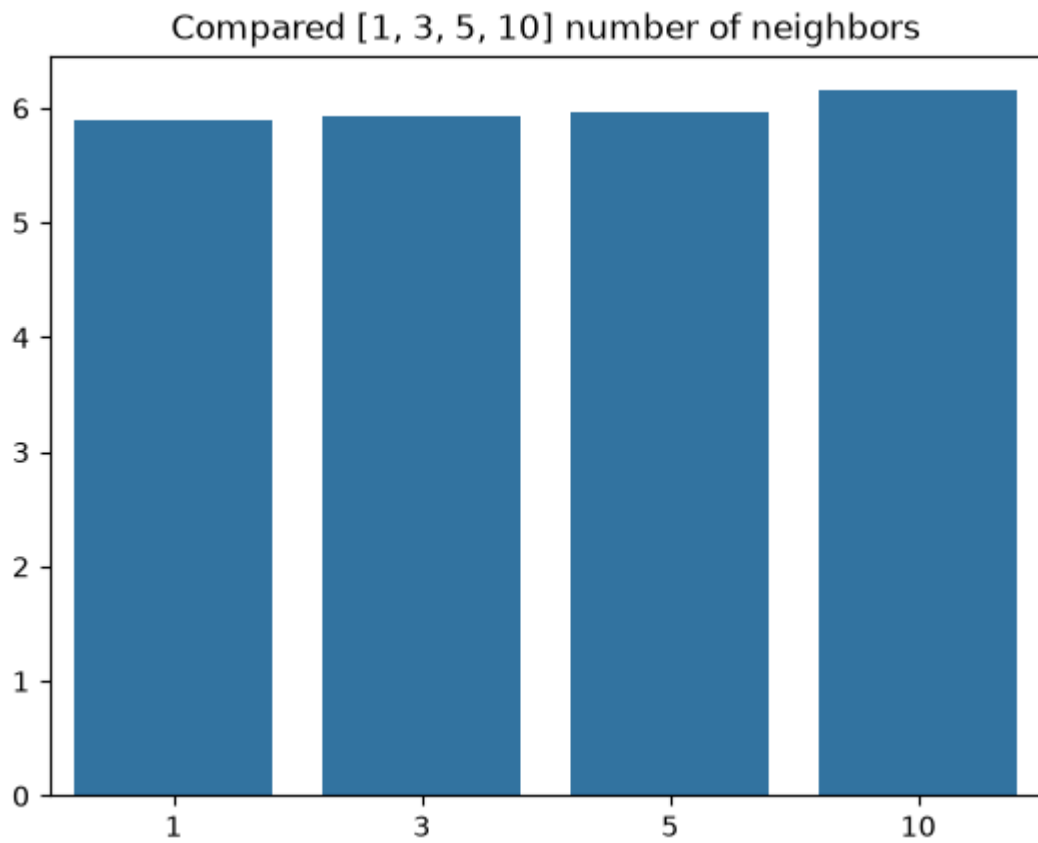
- Use n_neighbors 1, 3, 5, and 10 and print the predicted values of life satisfaction in Q10.
- Plot the results using a line plot

```
In [21]: n_neighbors = [1, 3, 5, 10]
        y_n_preds = []
        for n in n_neighbors:
            model = KNeighborsRegressor(n_neighbors=n).fit(X_train, y_train)
            y_pred = model.predict(
                pd.DataFrame(
                    {
                        "gdp_per_capita": [37_655.2]
                    }
                )
            )[0]
            y_n_preds.append(y_pred)
            print(f"{n=} : {y_pred=}")
```

```
n=1 : y_pred=np.float64(5.9)
n=3 : y_pred=np.float64(5.933333333333334)
n=5 : y_pred=np.float64(5.96)
n=10 : y_pred=np.float64(6.15)
```

```
In [22]: plt.title(f"Compared {n_neighbors} number of neighbors")
        sns.barplot(
```

```
x = n_neighbors,  
y = y_n_preds  
);
```



Because there are only 4 values and model is KNN, barplot is more suitable to compare predicted values between models with different `n_neighbors`